IS 5203 Lab #3 TCP

Daniela Salinas hyg998

[Lab adapted from University of South Carolina Network Tools and Protocols Lab Series]

In this lab, we'll investigate the behavior of the celebrated TCP protocol in detail. We'll do so by analyzing a trace of the TCP segments sent and received (1) between the VM and a web server and (2) between two mininet hosts. We'll study TCP's use of sequence and acknowledgement numbers for providing reliable data transfer; we'll see TCP's congestion control algorithm – slow start and congestion avoidance – in action.

Lab objectives:

By the end of this lab, students should be able to:

1. Capture a TCP transfer between your VM and a HTTP web server.
    - Extract basic information from TCP packets.
    - Analyze TCP traces.
2. Simulate a high-latency WAN and observe TCP congestion control behaviors.

Submission Instruction:

- **Save the file in .pdf format (Must be in PDF format, other formats including .doc will not be graded) with your name on the top of the document** and upload to the Blackboard item where you downloaded this instruction.

TCP Basics (20 pts)

In this part, you will configure a simple network topology for the experiments. You will use the iPerf3 tool to generate TCP traffic between the hosts in the network topology. You will run Wireshark to capture the TCP transfer.

Packet Capture and Basic Info

Evoke a terminal in your GUI.

In the terminal, use "sudo wireshark &" to run Wireshark. Start a Wireshark capture on the "eth0" interface.

In the terminal, we want to fetch a webpage from the remote web server. Use the below command to request for the website.

wget http://eu.httpbin.org

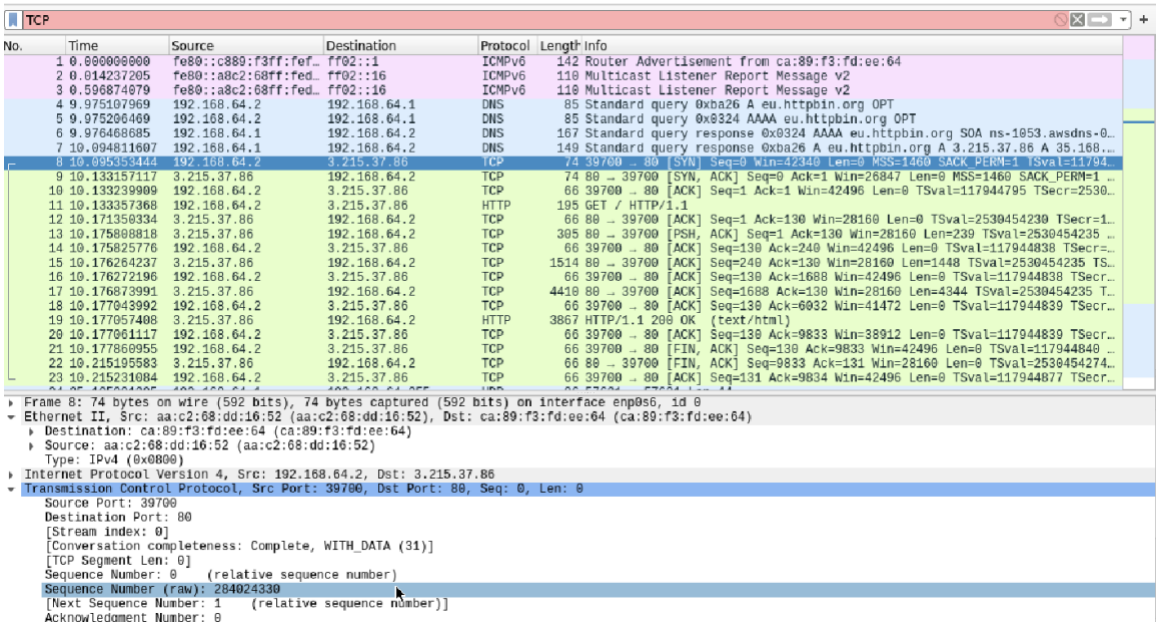
You can stop capturing when the download is finished. Since this lab is about TCP rather than HTTP, we can change Wireshark's configuration to only show the TCP segments (without further parsing the HTTP message within the segment). To have Wireshark do this, select

Analyze->Enabled Protocols. Then uncheck the HTTP box and select *OK*. You should now see a series of TCP segments sent between your computer and http://eu.httpbin.org.

Answer the following questions for the TCP segments:

Deliverable #1 (5 pts). What is the sequence number of the TCP SYN segment that is used to initiate the TCP connection between the client computer and eu.httpbin.org? (2 pts) What is it in the segment that identifies the segment as a SYN segment? (2 pts) Please also indicate with a screenshot. (1 pt)

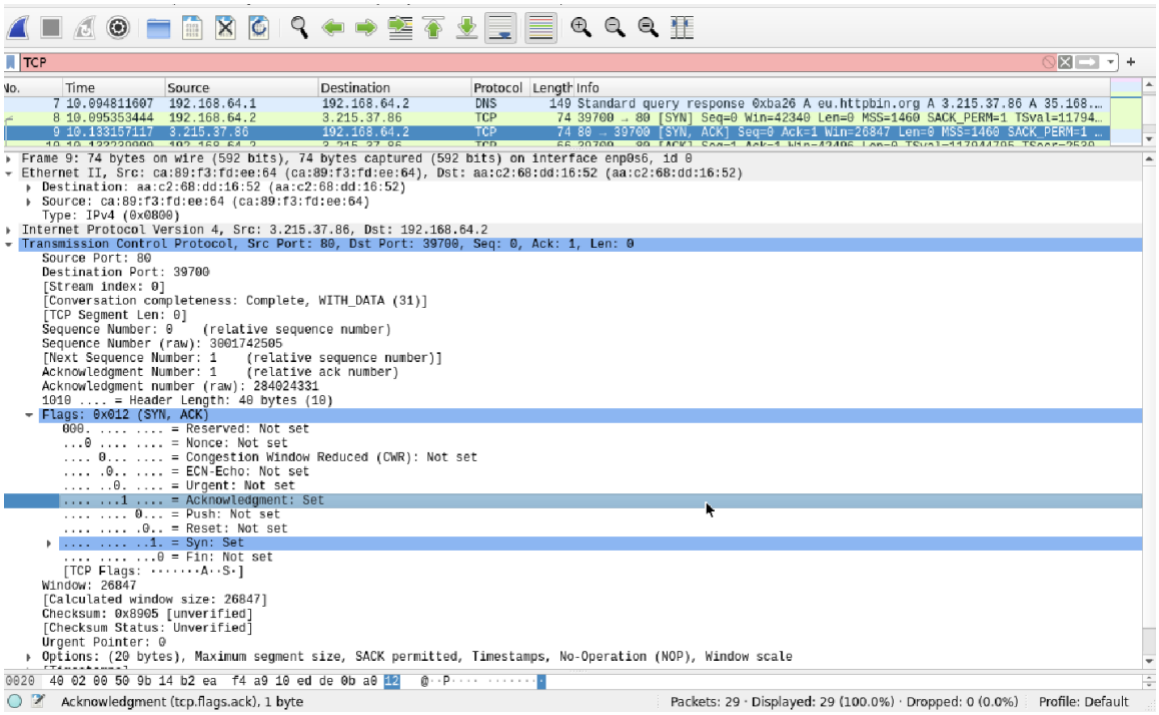
The sequence number of the TCP SYN segment is 0 since it is used to imitate the TCP connection between the client computer and the website. The Syn flag is set to 1 which indicates that this segment is a SYN segment.



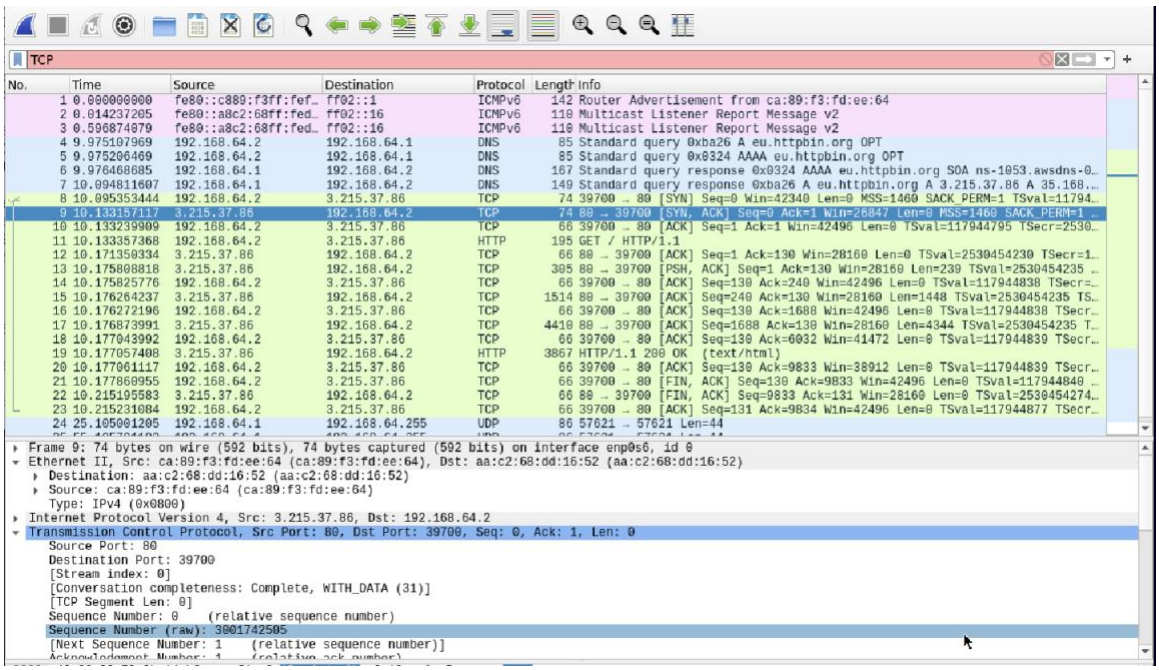
Deliverable #2 (8 pts). What is the sequence number of the SYNACK segment sent by eu.httpbin.org to the client computer in reply to the SYN? (2 pts) What is the value of the Acknowledgement field in the SYNACK segment? (2 pts) How did eu.httpbin.org determine that value? (2 pts) What is it in the segment that identifies the segment as a SYNACK segment? (2 pts)

The sequence number of the SYNACK segment sent by the website to the client computer in reply to the SYN is 0. The value of the acknowledgement field in the SYNACK segment is 1. The value of the acknowledgement field in the SYNACK segment is determined by the server of eu.httpbin.org. The server adds 1 to the initial sequence number of SYN segment from the client computer.

The initial sequence number of SYN segment from the client computer is 0, hence the value of the ACK field in the SYNACK segment is 1. A segment will be identified as a SYNACK segment if both SYN flag and Acknowledgement in the segment are set to 1.



Deliverable #3 (7 pts). Please paste a screenshot of the packet list (make sure that you include the Info column). (2 pts)



Please analyze how the server used seq# to tracks the data it sent. Also, demonstrate how your machine used ack# to track how much data it received. (5 pts)

Quit Wireshark to clean up and prepare for the next experiment.

TCP Congestion Control in Action (30 pts)

In this part, you will configure a simple network topology for the experiments. You will use the iPerf3 tool to generate TCP traffic between the hosts in the network topology. You will use plot_iperf.sh to generate diagrams.

Do the following to prepare for the analysis:

- **Setting up the Mininet topology**

We will use MiniEdit to set up the simple 2-host-2-switch topology. Launch a terminal and use "sudo ~/mininet/examples/miniedit.py" to launch MiniEdit.

In MiniEdit, please create the following topology:

In "Edit-Preferences," please select "start CLI." After the configuration, please click on the "Run" button on the bottom-left corner to start the topology.

Deliverable #4 (5 pts). Create the topology and use "pingall" to test network connectivity (1 pt).

```
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet>
```

Use "net" command in the mininet CLI to explore the network topology. Which interfaces were used by the link "s1-s2"? (2 pts)

```
mininet> net
h1 h1-eth0:s1-eth2
h2 h2-eth0:s2-eth2
s1 lo:  s1-eth1:s2-eth1 s1-eth2:h1-eth0
s2 lo:  s2-eth1:s1-eth1 s2-eth2:h2-eth0
mininet>
```

Use “h1 ifconfig” and “h2 ifconfig” to explore the IP address of the two hosts. (2 pts)

```
mininet> h1 ifconfig
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.0.1  netmask 255.0.0.0  broadcast 10.255.255.255
        inet6 fe80::8846:31ff:fe0b:2134  prefixlen 64  scopeid 0x20<link>
        ether 8a:46:31:0b:21:34  txqueuelen 1000  (Ethernet)
        RX packets 57  bytes 7334 (7.3 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 13  bytes 1006 (1.0 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

h2-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.0.2  netmask 255.0.0.0  broadcast 10.255.255.255
        inet6 fe80::f4a8:4cff:fe67:5e3b  prefixlen 64  scopeid 0x20<link>
        ether f6:a8:4c:67:5e:3b  txqueuelen 1000  (Ethernet)
        RX packets 58  bytes 7404 (7.4 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 13  bytes 1006 (1.0 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

- **Enabling TCP Congestion Control Algorithms**

Please invoke another terminal and execute the below command for this configuration. For most recent Linux distribution, two TCP Congestion Control algorithms are enabled on default, Reno and CUBIC. **To query the current algorithm being used by the system**, you can execute the command:

mininet@mininet-vm:~$ sysctl net.ipv4.tcp_congestion_control

We can use the following command **to see what congestion control algorithms are already loaded by the system**:

mininet@mininet-vm:~$ sysctl net.ipv4.tcp_available_congestion_control

You can use the following command to explore the available algorithm modules that we can load (those start with “tcp_”):

**mininet@mininet-vm:~$** ls /lib/modules/$(uname -r)/kernel/net/ipv4

Deliverable #5 (3 pts). What is the current TCP congestion algorithm used by the system (1 pt)?

Cubic

```
danny@ubuntu:~$ sysctl net.ipv4.tcp_congestion_control
net.ipv4.tcp_congestion_control = cubic
```

What are some available congestion algorithms that they system has loaded? (1 pt)

```
danny@ubuntu:~$ sysctl net.ipv4.tcp_available_congestion_control
net.ipv4.tcp_available_congestion_control = reno cubic
```

What are some other TCP congestion algorithm modules that are available to use? (1 pt)

```
danny@ubuntu:~$ ls /lib/modules/$(uname -r)/kernel/net/ipv4
ah4.ko           ip_tunnel.ko  tcp_highspeed.ko  tcp_westwood.ko
esp4.ko          ip_vti.ko     tcp_htcp.ko       tcp_yeah.ko
esp4_offload.ko  netfilter     tcp_hybla.ko      tunnel4.ko
fou.ko           raw_diag.ko   tcp_illinois.ko   udp_diag.ko
gre.ko           tcp_bbr.ko    tcp_lp.ko         udp_tunnel.ko
inet_diag.ko     tcp_bic.ko    tcp_nv.ko         xfrm4_tunnel.ko
ipcomp.ko        tcp_cdg.ko    tcp_scalable.ko
ip_gre.ko        tcp_dctcp.ko  tcp_vegas.ko
ipip.ko          tcp_diag.ko   tcp_veno.ko
```

To prepare for our experiment, we also want to enable TCP Vegas algorithm.

To load TCP Vegas on the host machine, please run:

mininet@mininet-vm:~$ sudo modprobe tcp_vegas

You can check if the Vegas algorithm is loaded by running the following command again:

mininet@mininet-vm:~$ sysctl net.ipv4.tcp_available_congestion_control

We will create three folders to store the experiment results:

mininet@mininet-vm:~$ mkdir cubic

mininet@mininet-vm:~$ mkdir vegas

mininet@mininet-vm:~$ mkdir reno

- **Configure the links: emulating 1 Gbps high-latency WAN with packet loss**

To simulate the real network environment, we can configure the link between s1 and s2 with a bandwidth of 1Gbps and a packet loss rate of 0.01%. We also would like to add a 30 ms delay to this link to simulate a busy network.

mininet@mininet-vm:~$ sudo tc qdisc add dev s1-eth2 root handle 1: netem loss 0.01% delay 30ms

Then, please use the following command to set the bandwidth to 1 Gbps on switch S1's s1-eth2 interface.

mininet@mininet-vm:~$ sudo tc qdisc add dev s1-eth2 parent 1: handle 2: tbf rate 1gbit burst 500000 limit 2500000

Next, we want to configure the TCP buffer of the two mininet hosts.

In mininet CLI, use:

mininet> xterm h1

mininet> xterm h2

to evoke the terminal of each host.

For <u>each host (i.e., both h1 and h2)</u>, configure the TCP read and write buffers with the following two commands in their corresponding terminals:

root@mininet-vm:/home/mininet# sysctl -w net.ipv4.tcp_rmem='10240 87380 150000000'

root@mininet-vm:/home/mininet# sysctl -w net.ipv4.tcp_wmem='10240 87380 150000000'

- **Using iPerf3 to generate traffic (TCP CUBIC)**

On default, Linux uses TCP CUBIC as the congestion control algorithm. So we first examine the CUBIC algorithm.

Please launch the iperf server <u>on h2</u>:

root@mininet-vm:/home/mininet# iperf3 -s

<u>On h1,</u> we first change the directory to the "cubic" folder we created earlier.

root@mininet-vm:/home/mininet# cd cubic

We then use iPerf3 to generate **<u>30-second TCP traffic</u>** to h2 (10.0.0.2). We will store the results in a json file. In the command, we explicitly request to send the traffic with TCP CUBIC algorithm.

root@mininet-vm:/home/mininet/cubic# iperf3 -c 10.0.0.2 -t 30 -C cubic -J > cubic.json

Once the transmission is finished, run the following command to plot the results into the results folder.

root@mininet-vm:/home/mininet/cubic# plot_iperf.sh cubic.json

In h2's xterm, stop the iperf3 server by Ctrl+C. We will launch the server in the next set of experiment.

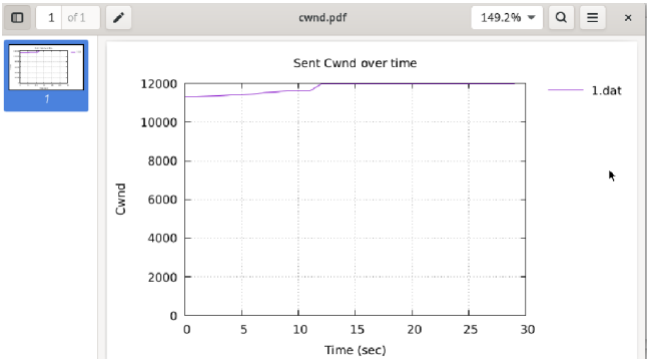
Open a Linux terminal and navigate to the ~/cubic/results folder. Use Evince to open the cwnd.pdf file.

mininet@mininet-vm:~$ cd ~/cubic/results

mininet@mininet-vm:~/cubic/results$ evince cwnd.pdf

Deliverable #6 (7 pts). Paste a screenshot of the cwnd diagram. (1 pt) Please describe how cwnd evolved over time. (3 pts) What happened when there was a congestion? (2 pts) Do you observe a cubic increase after the multiplicative decrease? (1 pt)

Yes, there's cubic increase after multiplicative decrease.



- **Using iPerf3 to generate traffic (TCP Vegas)**

We now want to examine TCP Vegas's behavior. On h2, let's launch the server again.

root@mininet-vm:/home/mininet# iperf3 -s

On h1's xterm, we want to change directory to the "vegas" folder we created earlier.

root@mininet-vm:/home/mininet/cubic# cd ../vegas

We will change the congestion algorithm on h1. Please run the following command to switch the algorithm.

root@mininet-vm:/home/mininet/vegas# sysctl -w net.ipv4.tcp_congestion_control=vegas

We then use iPerf3 to generate 30-second TCP traffic to h2 (10.0.0.2). We will store the results in a json file. In the command, we explicitly request to send the traffic with TCP Vegas algorithm.

root@mininet-vm:/home/mininet/vegas# iperf3 -c 10.0.0.2 -t 30 -C vegas -J > vegas.json

Once the transmission is finished, run the following command to plot the results into the results folder.

root@mininet-vm:/home/mininet/vegas# plot_iperf.sh vegas.json

In h2's xterm, stop the iperf3 server by Ctrl+C. We will launch the server in the next set of experiment.

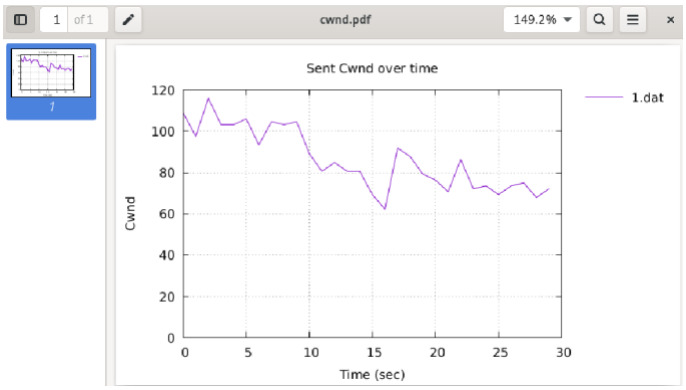
Open a Linux terminal and navigate to the ~/vegas/results folder. Use Evince to open the cwnd.pdf file.

mininet@mininet-vm:~$ cd ~/vegas/results

mininet@mininet-vm:~/vegas/results$ evince cwnd.pdf

Deliverable #7 (6 pts). Paste a screenshot of the cwnd diagram. (1 pt) Please describe how cwnd evolved over time. (3 pts) What happened when there was a congestion? (2 pts)

Cwnd started off high but decreased over time. Overall it's volatile and after everytime high decrease there's a steady increase.



- **Using iPerf3 to generate traffic (TCP Reno)**

We now want to examine TCP Reno's behavior. On h2, let's launch the server again.

root@mininet-vm:/home/mininet# iperf3 -s

On h1's xterm, we want to change directory to the "reno" folder we created earlier.

root@mininet-vm:/home/mininet/vegas# cd ../reno

We will change the congestion algorithm on h1. Please run the following command to switch the algorithm.

root@mininet-vm:/home/mininet/reno# sysctl -w net.ipv4.tcp_congestion_control=reno

We then use iPerf3 to generate 30-second TCP traffic to h2 (10.0.0.2). We will store the results in a json file. In the command, we explicitly request to send the traffic with TCP Reno algorithm.

root@mininet-vm:/home/mininet/reno# iperf3 -c 10.0.0.2 -t 30 -C reno -J > reno.json

Once the transmission is finished, run the following command to plot the results into the results folder.

root@mininet-vm:/home/mininet/reno# plot_iperf.sh reno.json

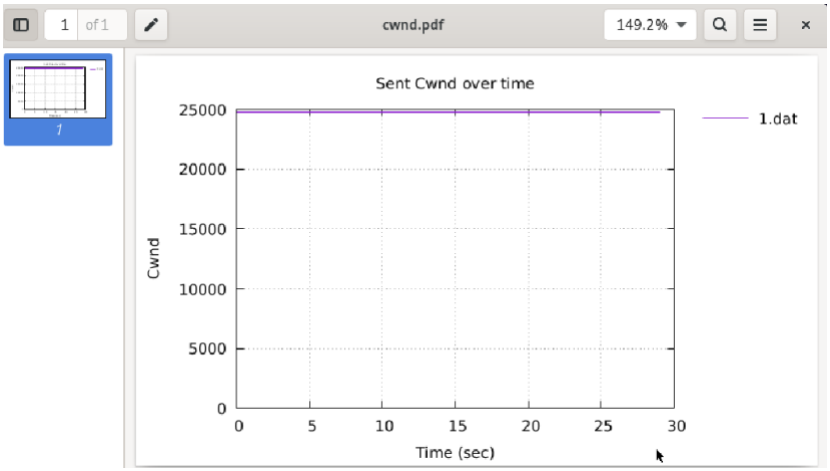
In h2's xterm, stop the iperf3 server by Ctrl+C.

Open a Linux terminal and navigate to the ~/reno/results folder. Use Evince to open the cwnd.pdf file.

mininet@mininet-vm:~$ cd ~/reno/results

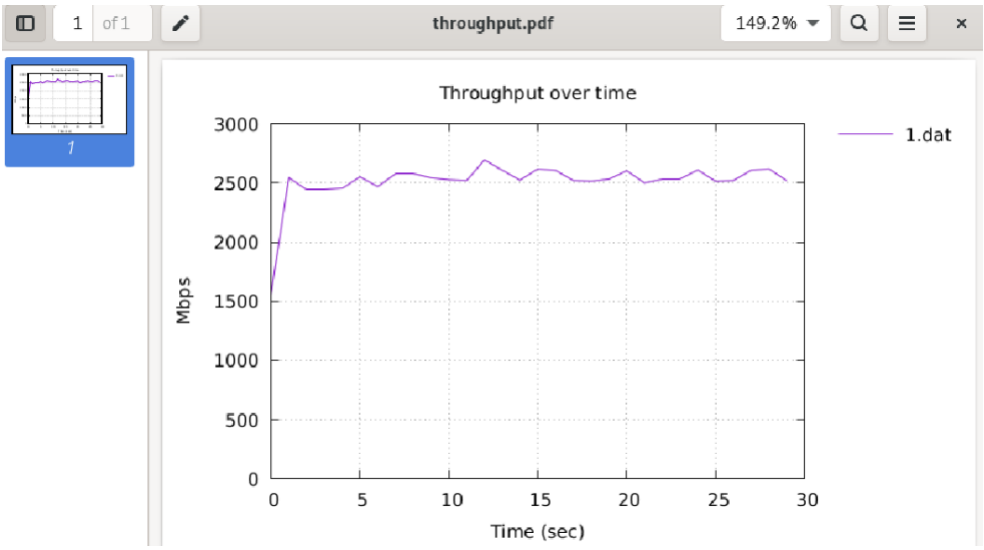
mininet@mininet-vm:~/reno/results$ evince cwnd.pdf

<u>Deliverable #8 (6 pts).</u> Paste a screenshot of the cwnd diagram. (1 pt) Please describe how cwnd evolved over time. (3 pts) What happened when there was a congestion? (2 pts)

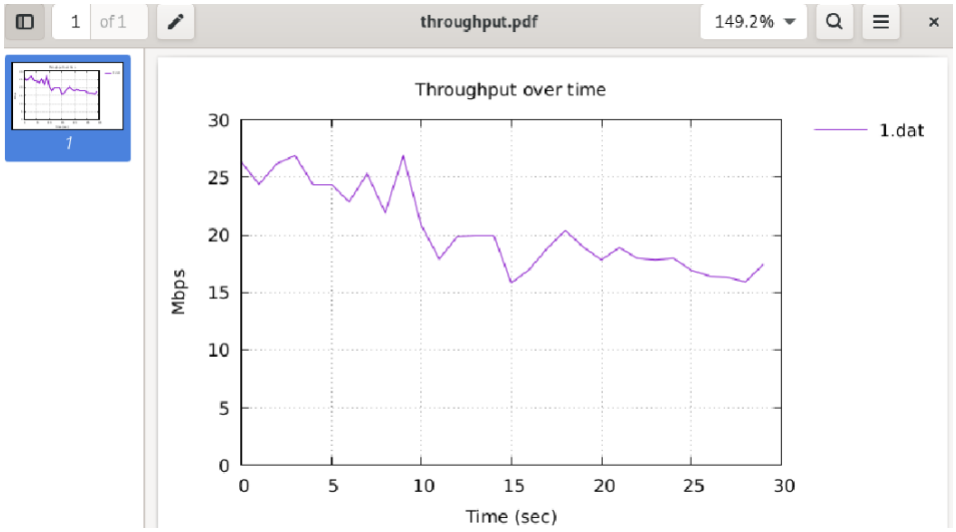


<u>Deliverable #9 (3 pts).</u> Paste the screenshots of the following three throughput files:

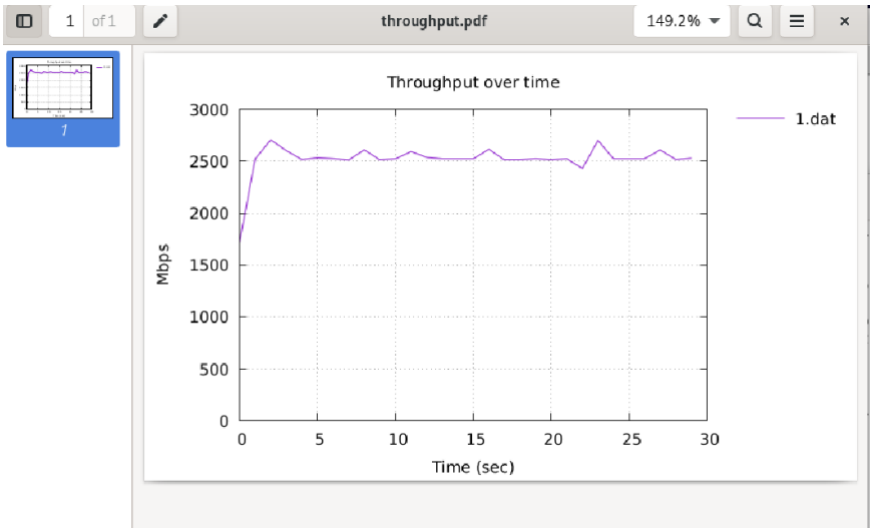
mininet@mininet-vm:~ $ evince ~/cubic/results/throughput.pdf



mininet@mininet-vm:~ $ evince ~/vegas/results/throughput.pdf



mininet@mininet-vm:~ $ evince ~/reno/results/throughput.pdf



Comparing the three algorithms, which algorithm do you prefer for our emulated high-latency WAN connection with packet loss? Why?

I would choose option B (vegas) since the algorithm is correct for high latency WAN connection.